CAA Lab Assignment 4

```
struct CPU {
  uint8_t R[4];
  uint8_t ACC;
  uint8_t PC;
  uint8_t Z, C;
};

struct IFEX {
  uint8_t IR;
  bool valid;
};

struct CacheLine {
  bool valid;
  uint8_t tag;
  uint8_t data;
};

CPU cpu;
IFEX ifex;
CacheLine cache[CACHE_LINES];
uint8_t IMEM[MEM_SIZE];
uint8_t DMEM[MEM_SIZE];

int cycles = 0;
int instructions = 0;
int stalls = 0;
int forwardings = 0;
int cache_hits = 0;
int cache_misses = 0;
bool jump_taken = false;

void reset_cpu() {
  memset(&cpu, 0, sizeof(cpu));
  memset(&ifex, 0, sizeof(ifex));
  memset(cache, 0, sizeof(cache));
}
```

```
void load_program(const string &fname) {
  ifstream fin(fname);
  if (!fin) {
    cout << "Cannot open program file\n";
    exit(1);
  }
  int addr = 0;
  unsigned int x;
  while (fin >> hex >> x)
    IMEM[addr++] = (uint8_t)x;
}

bool is_load(uint8_t ir) {
  return ((ir >> 4) & 0xF) == 0xD;
}

bool writes_register(uint8_t ir) {
  int op = (ir >> 4) & 0xF;
  return (op == 0x0 || op == 0x1 || op == 0x2 || op == 0x3 ||
          (op >= 0x4 && op <= 0x7));
}

uint8_t cache_read(uint8_t addr) {
  int index = addr % CACHE_LINES;
  uint8_t tag = addr / CACHE_LINES;

  if (cache[index].valid && cache[index].tag == tag) {
    cache_hits++;
    return cache[index].data;
  }

  cache_misses++;
  cycles += CACHE_MISS_PENALTY;

  cache[index] = {true, tag, DMEM[addr]};
  return DMEM[addr];
}

void cache_write(uint8_t addr, uint8_t val) {
  int index = addr % CACHE_LINES;
  uint8_t tag = addr / CACHE_LINES;

  DMEM[addr] = val;
```

```
  cache[index] = {true, tag, val};
}

uint8_t alu(uint8_t a, uint8_t b, int op) {
  int res = 0;
  if (op == 0x4) res = a + b;
  if (op == 0x5) res = a - b;
  if (op == 0x6) res = a * b;
  if (op == 0x7) res = (b == 0 ? 0 : a / b);

  cpu.Z = (res == 0);
  cpu.C = (res > 255);
  return res & 0xFF;
}

void execute_stage() {
  if (!ifex.valid) return;

  uint8_t ir = ifex.IR;
  int op = (ir >> 4) & 0xF;
  int operand = ir & 0xF;

  instructions++;

  switch (op) {
  case 0x1: cpu.ACC = cpu.R[operand]; break;
  case 0x2: cpu.R[operand] = cpu.ACC; break;
  case 0x3: cpu.ACC = operand; break;
  case 0x4: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x4); break;
  case 0x5: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x5); break;
  case 0x6: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x6); break;
  case 0x7: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x7); break;
  case 0x0: cpu.R[operand] = cache_read(operand); break;
  case 0xE: cache_write(operand, cpu.R[operand]); break;
  case 0xA:
    cpu.PC = operand;
    jump_taken = true;
    break;
  }
}

void fetch_stage() {
  ifex.IR = IMEM[cpu.PC++];
  ifex.valid = true;
}
```

CAA Lab Assignment 5

```
int prediction_mode = ONE_BIT_BP;

struct CPU {
  unsigned char R[4];
  unsigned char ACC;
  unsigned char PC;
  unsigned char IR;
  unsigned char Z, C;
};

struct IFEX {
  unsigned char IR;
  unsigned char PC;
  int valid;
  int predicted_taken;
};

struct BPEntry {
  int valid;
  int prediction;
};

unsigned char IMEM[MEM_SIZE];
unsigned char DMEM[MEM_SIZE];
CPU cpu;
IFEX ifex;
BPEntry bp_table[BP_SIZE];

int cycles = 0;
int instructions = 0;
int total_branches = 0;
int correct_predictions = 0;
int mispredictions = 0;
int branch_penalty_cycles = 0;

void reset_cpu() {
  memset(&cpu, 0, sizeof(cpu));
  memset(&ifex, 0, sizeof(ifex));
  memset(bp_table, 0, sizeof(bp_table));
  cpu.IE = 1;
  cpu.R[1] = 1;
  cpu.R[2] = 0;
}

void load_program(const string& fname) {
  ifstream fin(fname);
  if (!fin) {
    cout << "Cannot open program file\n";
    exit(1);
  }
  int addr = 0;
  unsigned int x;
  while (fin >> hex >> x)
    IMEM[addr++] = (unsigned char)x;
  fin.close();
}

int is_branch(unsigned char ir) {
  unsigned char op = (ir >> 4) & 0xF;
  return (op == 0xA || op == 0xB);
}

int branch_taken(unsigned char ir) {
  unsigned char op = (ir >> 4) & 0xF;
  return (op == 0xA) || (op == 0xB && cpu.Z);
}

int bp_index(unsigned char pc) {
  return pc % BP_SIZE;
}

int predict_branch(unsigned char pc) {
  if (prediction_mode == ONE_BIT_BP) {
    int idx = bp_index(pc);
    if (bp_table[idx].valid)
      return bp_table[idx].prediction;
  }
  return 0;
}

void update_predictor(unsigned char pc, int taken) {
```

```
  if (prediction_mode != ONE_BIT_BP)
    return;

  int idx = bp_index(pc);
  bp_table[idx].valid = 1;

  if (taken && bp_table[idx].counter < 3)
    bp_table[idx].counter++;
  else if (!taken && bp_table[idx].counter > 0)
    bp_table[idx].counter--;
}

uint8_t alu(uint8_t a, uint8_t b, int op) {
  int res = 0;
  if (op == 0x4) res = a + b;
  if (op == 0x5) res = a - b;
  if (op == 0x6) res = a * b;
  if (op == 0x7) res = (b == 0 ? 0 : a / b);

  cpu.Z = (res == 0);
  cpu.C = (res > 255);
  return res & 0xFF;
}

int flush_pipeline = 0;

void fetch_stage() {
  ifex.IR = IMEM[cpu.PC++];
  ifex.PC = cpu.PC;
  ifex.valid = 1;

  if (is_branch(ifex.IR)) {
    int pred = predict_branch(cpu.PC);
    ifex.predicted_taken = pred;
    unsigned char operand = ifex.IR & 0xF;
    cpu.PC = pred ? operand : (ifex.PC + 1);
  }
}

void execute_stage() {
  if (!ifex.valid) return;
  unsigned char ir = ifex.IR;
  unsigned char op = (ir >> 4) & 0xF;
  unsigned char operand = ir & 0xF;
  instructions++;

  switch (op) {
  case 0x3:
    cpu.ACC = operand;
    break;
  case 0x4:
    cpu.ACC += cpu.R[operand];
    break;
  case 0x5:
    cpu.ACC -= cpu.R[operand];
    break;
  case 0x6:
    cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x6);
    break;
  case 0x7:
    cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x7);
    break;
  case 0x0:
    cpu.R[operand] = DMEM[operand];
    break;
  case 0xE:
    DMEM[operand] = cpu.R[operand];
    break;
  case 0xA:
    cpu.PC = operand;
    jump_taken = true;
    break;
  }
}
```

CAA Lab Assignment 6

```
struct CPU {
  unsigned char R[4];
  unsigned char ACC;
  unsigned char PC;
  unsigned char Z, C;
  unsigned char EPC;
  unsigned char CAUSE;
  unsigned char IE;
};

struct IFEX {
  unsigned char IR;
  unsigned char PC;
  int valid;
  int predicted_taken;
};

struct BPEntry {
  int valid;
  int counter;
};

void reset_cpu() {
  memset(&cpu, 0, sizeof(cpu));
  memset(&ifex, 0, sizeof(ifex));
  memset(bp_table, 0, sizeof(bp_table));
  cpu.IE = 1;
  cpu.R[1] = 1;
  cpu.R[2] = 0;
}

void load_program(const char* file) {
  ifstream fin(file);
  if (!fin) {
    cout << "Program load error\n";
    exit(1);
  }
  int i = 0;
  unsigned int x;
  while (fin >> hex >> x)
    IMEM[i++] = (unsigned char)x;
  fin.close();
}

int bp_index(unsigned char pc) {
  return pc % BP_SIZE;
}

int predict_branch(unsigned char pc) {
  int idx = bp_index(pc);
  if (bp_table[idx].valid)
    return bp_table[idx].counter >= 2;
  return 0;
}

void update_predictor(unsigned char pc, int taken) {
  int idx = bp_index(pc);
  bp_table[idx].valid = 1;

  if (taken && bp_table[idx].counter < 3)
    bp_table[idx].counter++;
  else if (!taken && bp_table[idx].counter > 0)
    bp_table[idx].counter--;
}

uint8_t alu(uint8_t a, uint8_t b, int op) {
  int res = 0;
  if (op == 0x4) res = a + b;
  if (op == 0x5) res = a - b;
  if (op == 0x6) res = a * b;
  if (op == 0x7) res = (b == 0 ? 0 : a / b);
  cpu.Z = (res == 0);
  cpu.C = (res > 255);
  return res & 0xFF;
}

int interrupt_pending() {
  return (cpu.IE && (cycles % TIMER_PERIOD == 0));
}

void raise_interrupt(int cause) {
```

```
  cpu.EPC = cpu.PC;
  cpu.CAUSE = cause;
  cpu.IE = 0;
  ifex.valid = 0;
  cpu.PC = INT_HANDLER;
  interrupts++;
}

void raise_exception(int cause) {
  cpu.EPC = cpu.PC;
  cpu.CAUSE = cause;
  cpu.IE = 0;
  ifex.valid = 0;
  cpu.PC = INT_HANDLER;
  exceptions++;
}

void fetch_stage() {
  ifex.IR = IMEM[cpu.PC++];
  ifex.PC = cpu.PC;
  ifex.valid = 1;
  unsigned char op = (ifex.IR >> 4) & 0xF;
  if (op == OP_JMP || op == OP_JZ) {
    int pred = predict_branch(cpu.PC);
    ifex.predicted_taken = pred;
    unsigned char operand = ifex.IR & 0xF;
    cpu.PC = pred ? operand : cpu.PC + 1;
  }
}

void execute_stage() {
  if (!ifex.valid) return;
  unsigned char ir = ifex.IR;
  unsigned char op = (ir >> 4) & 0xF;
  unsigned char operand = ir & 0xF;
  unsigned char arg = ir & 0xF;
  instructions++;

  switch (op) {
  case OP_MOVI:
    cpu.ACC = arg;
    break;
  case OP_LOAD:
    cpu.ACC = cpu.R[arg];
    break;
  case OP_STORE:
    cpu.R[arg] = cpu.ACC;
    break;
  case OP_ADD:
    cpu.ACC += cpu.R[arg];
    break;
  case OP_SUB:
    cpu.ACC -= cpu.R[arg];
    break;
  case OP_DIV:
    if (cpu.R[arg] == 0) {
      raise_exception(1);
      return;
    }
    cpu.ACC /= cpu.R[arg];
    break;
  case OP_CMP:
    cpu.Z = (cpu.ACC == cpu.R[arg]);
    break;
  case OP_JMP:
    cpu.PC = operand;
    jump_taken = true;
    break;
  }
}
```

CAA Test 1

```
enum class State {
  IDLE, LOCK, FILL, WASH, DRAIN, UNLOCK, BUZZ, ERROR_STATE
};

enum class Mode {
  QUICK = 2, NORMAL = 4, HEAVY = 6
};

struct Inputs {
  bool start = false;
  bool door_closed = true;
  bool water_ok = false;
  bool overload = false;
};

struct Outputs {
  bool motor = false;
  bool valve = false;
  bool door_lock = false;
  bool buzzer = false;
};

string stateName(State s) {
  switch (s) {
  case State::IDLE: return "IDLE";
  case State::LOCK: return "LOCK";
  case State::FILL: return "FILL";
  case State::WASH: return "WASH";
  case State::DRAIN: return "DRAIN";
  case State::UNLOCK: return "UNLOCK";
  case State::BUZZ: return "BUZZ";
  case State::ERROR_STATE: return "ERROR";
  }
  return "UNKNOWN";
}

Outputs getOutputs(State s) {
  switch (s) {
  case State::LOCK: return {false, false, true, false};
  case State::FILL: return {false, true, true, false};
  case State::WASH: return {true, false, true, false};
  case State::DRAIN: return {false, false, true, false};
  case State::UNLOCK: return {false, false, false, false};
  case State::BUZZ: return {false, false, false, true};
  case State::ERROR_STATE: return {false, false, true, true};
  default: return {false, false, false, false};
  }
}

class WashingMachineController {
public:
  explicit WashingMachineController(Mode mode)
      : currentState(State::IDLE), mode(mode), stateTimer(0),
        washCycles(0), totalCycles(static_cast<int>(mode)) {}

  bool tick(int cycleNum, const Inputs& inputs) {
    if (checkInterrupt(inputs)) {
      currentState = State::ERROR_STATE;
    }

    Outputs out = getOutputs(currentState);

    if (currentState == State::ERROR_STATE) {
      return false;
    }

    if (currentState == State::BUZZ) {
      return false;
    }

    transitionState(inputs);
    return true;
  }

  State getState() const { return currentState; }

private:
  State currentState;
  Mode mode;
  int stateTimer;
  int washCycles;
```

```
  int totalCycles;

  bool checkInterrupt(const Inputs& in) {
    if (currentState == State::ERROR_STATE) return false;
    if (currentState == State::IDLE) return false;
    if (currentState == State::BUZZ) return false;

    if (in.overload) return true;
    if (!in.door_closed) return true;

    return false;
  }

  void transitionState(const Inputs& in) {
    switch (currentState) {
    case State::IDLE:
      if (in.start && in.door_closed)
        currentState = State::LOCK;
      break;

    case State::LOCK:
      stateTimer++;
      if (stateTimer >= 2) {
        stateTimer = 0;
        currentState = State::FILL;
      }
      break;

    case State::FILL:
      if (in.water_ok)
        currentState = State::WASH;
      break;

    case State::WASH:
      washCycles++;
      if (washCycles >= totalCycles) {
        washCycles = 0;
        currentState = State::DRAIN;
      }
      break;

    case State::DRAIN:
      currentState = State::UNLOCK;
      break;

    case State::UNLOCK:
      stateTimer++;
      if (stateTimer >= 2) {
        stateTimer = 0;
        currentState = State::BUZZ;
      }
      break;

    case State::BUZZ:
      break;

    case State::ERROR_STATE:
      break;
    }
  }
};
```

CAA Test 2

```
int main() {
    std::ofstream trace("cpu_trace.txt");
    int modeInput, cycles;
    std::cout << "Mode (0=DEFROST 1=HEAT 2=GRILL): ";
    std::cin >> modeInput;
    Mode mode = (Mode)modeInput;
    std::cout << "Number of cycles: ";
    std::cin >> cycles;

    State state = IDLE;
    uint8_t timer = 0;

    trace << "Cycle | State | Inputs | Outputs | Display\n";
    trace << std::string(65, '-') << "\n";

    for (int cycle = 1; cycle ≤ cycles; cycle++) {
        int s, st, d, f, t;
        std::cin >> s >> st >> d >> f >> t;
        bool START = s, STOP = st, DOOR = d, FOOD = f, TEMP = t;

        if (state ≠ ERROR_STATE) {
            if (state ≠ IDLE && state ≠ BUZZ) {
                if (TEMP) {
                    state = ERROR_STATE;
                } else if (!DOOR) {
                    state = PAUSE;
                } else if (STOP && state == RUN) {
                    state = PAUSE;
                }
            }
            switch (state) {
            case IDLE:
                if (START && DOOR && FOOD) {
                    state = CHECK;
                    timer = modeCycles[mode];
                }
                break;
            case CHECK:
                state = RUN;
                break;
            case RUN:
                {
                    ALUResult res = alu_sub(timer, 1);
                    timer = res.value;
                    if (res.Z) state = COMPLETE;
                }
                break;
            case PAUSE:
                if (START && DOOR) {
                    state = RUN;
                }
                break;
            case COMPLETE:
                state = BUZZ;
                break;
            case BUZZ:
                state = IDLE;
                timer = 0;
                break;
            case ERROR_STATE:
                break;
            }
        }
        bool heater    = (state == RUN);
        bool turntable = (state == RUN);
        bool light     = (state == CHECK || state == RUN || state ==
PAUSE || state == COMPLETE || state == ERROR_STATE);
        bool buzzer    = (state == BUZZ || state == ERROR_STATE);

        logLine(trace, cycle, state, DOOR, FOOD, TEMP, heater,
turntable, light, buzzer, mode, timer);
    }
    trace.close();
    std::cout << "\nTrace saved to cpu_trace.txt\n";
    return 0;
}
```